# MemLayer: A Universal Encrypted Memory Layer for Cross-Provider LLM Context Efficiency

Abhishek Shekhar
https://github.com/Abhi183/ai-memory-layer

## Abstract

Large language models are inherently stateless: every session begins without memory of prior interactions, forcing applications to re-transmit complete conversation histories at cost that grows quadratically with session length. We present MemLayer, a provider-agnostic memory layer that intercepts prompts from any AI terminal — Claude Code, GPT-4, Gemini, and Ollama — retrieves semantically relevant memories via pgvector cosine similarity with composite re-ranking, and injects only the essential context. We describe the architecture, the composite retrieval algorithm (combining semantic similarity, recency decay, and importance scoring), the per-user AES-256-GCM encryption model, and a projected cost analysis based on publicly available provider pricing. Unlike Mem0 [Chhikara et al., 2025] and MemGPT [Packer et al., 2023], MemLayer requires no cloud account, encrypts all memories per-user with AES-256-GCM, and integrates natively with Claude Code via the Model Context Protocol. Under the assumption that memory injection reduces per-query context from a full session history ($T_{\text{full}}$ = 15,000 tokens) to a targeted retrieval set ($T_{\text{aug}}$ = 1,000 tokens), the projected Token Compression Ratio is 93.3% — consistent with the 90% reduction reported by Mem0 on the LOCOMO benchmark [Chhikara et al., 2025]. Empirical evaluation on real conversational data is ongoing. All code is released at https://github.com/Abhi183/ai-memory-layer.

## 1 Introduction

Every session starts from zero. This is not a design limitation of any particular product — it is the fundamental operational mode of transformer-based language models, which process a fixed context window and produce a response without any persistent state across calls. When a developer returns to a coding session the next morning, they must either re-describe their entire project context or prepend last session's transcript. Both strategies pay for information that was already paid for.

The cost structure is quadratic, not linear. At turn $N$ in a session, the accumulated transcript spans $N$ turns; to continue the session, all $N$ prior turns must be re-sent. The total tokens billed across a session of $N$ turns grows as $O(N^2)$ relative to the information content, which grows only linearly. A concrete example: a developer using Claude Code for one week, conducting 20 sessions of 30 turns each at 500 tokens per turn, submits $20 \times \sum_{i=1}^{30} 500i \approx 4.65$ million tokens. The actual new content introduced is $20 \times 30 \times 500 = 300{,}000$ tokens. The remaining 4.35 million tokens — 93.5% of the bill — are redundant history.

Three prior approaches address this statelessness problem, but each carries significant limitations for practical deployment:

1. **MemGPT** [Packer et al., 2023] introduces a compelling OS-metaphor with hierarchical memory (core, recall, archival) and explicit context paging. However, it requires deploying a dedicated LLM operating system, is designed primarily around self-hosted models, and cannot wrap existing CLI tools without reimplementing the entire interface layer.

2. **Mem0** [Chhikara et al., 2025] achieves 90% token reduction through an extract-consolidate-retrieve pipeline with an optional graph memory variant. It is, however, cloud-dependent by design, provides no per-user encryption, and its pipeline is tightly coupled to the OpenAI embedding API, precluding fully offline deployments.

3. **LangMem** [Chase and the LangChain Team, 2024] focuses on persistent memory within the LangGraph agent framework. Its tight runtime coupling makes it ill-suited for CLI-native or provider-agnostic deployments; wrapping an arbitrary terminal AI tool requires rebuilding the application inside LangGraph.

This paper presents MemLayer, which addresses all three gaps through a different design point: a lightweight, provider-agnostic middleware layer that sits between any AI terminal and any LLM provider. We investigate three specific research questions:

1. What token compression ratio does semantic retrieval project under realistic session-length assumptions, and how does it translate to provider-specific cost savings?

2. Does composite scoring (combining cosine similarity, recency, and importance) provide a theoretically sound advantage over pure cosine similarity, and what weight configuration is appropriate?

3. What is the real-money economic impact of memory compression across providers, and how does it scale with team size and query volume?

# 2 Background

## 2.1 Long-Context LLMs

Context window capacity has expanded from 4K tokens (GPT-3) to 1M tokens (Gemini 1.5 Pro) within three years [Vaswani et al., 2017]. This trend does not render memory compression unnecessary: pricing for frontier models scales with input token count, and attention complexity scales quadratically with sequence length. At $3 per million tokens, a 15,000-token session history costs $0.045 per query. Memory injection targeting 1,000 tokens reduces this to $0.003 per query — a 93.3% projected reduction, derived directly from the provider's published pricing and the token counts described in Section 4.

## 2.2 MemGPT

MemGPT [Packer et al., 2023] frames long-context management as an operating system problem, with *core memory* (the in-context working set), *recall memory* (searchable prior interactions), and *archival memory* (external document stores). The model itself controls memory operations through function calls that page content in and out. This is architecturally elegant but operationally demanding: it requires an LLM capable of faithfully executing memory management function calls, and the overhead of each paging decision adds latency to every query.

## 2.3 Mem0

Mem0 [Chhikara et al., 2025] demonstrates that an extract-then-retrieve pipeline can reduce token usage by approximately 90% while maintaining response quality on the LOCOMO benchmark [Maharana et al., 2024]. Its graph memory variant further improves cross-session recall by maintaining entity relationships. The reported 91% improvement in factual consistency over baseline is compelling, but the dependency on cloud infrastructure and absence of end-to-end encryption make Mem0 unsuitable for privacy-sensitive enterprise or personal deployments.

## 2.4 LangMem

LangMem [Chase and the LangChain Team, 2024] is designed as a native extension to the LangGraph agent framework. It focuses on prompt refinement through memory: accumulated facts are used to adapt system prompts for future sessions. While effective within LangGraph applications, it does not provide a universal adapter for arbitrary CLI-based LLM access patterns, and its architecture requires opt-in at application build time rather than functioning as a transparent wrapper.

## 2.5 KV Cache Compression

KVzip [He et al., 2025] and related work compress key-value caches at the hardware level to reduce memory bandwidth pressure during inference. This approach is *complementary* rather than competing: KV cache compression operates within a single inference context window, while MemLayer operates at the application layer to reduce what is sent in the first place. The two can be combined.

## 2.6 Retrieval-Augmented Generation

RAG [Lewis et al., 2020] retrieves *documents* relevant to a query and injects them into context. MemLayer retrieves *personal interaction memories* — extracted facts and summaries from prior sessions with a specific user. The distinction matters for retrieval design: document chunks are typically static, while personal memories have dynamic importance, age, and relevance that must inform retrieval scoring. A-MEM [Xu et al., 2025] explores Zettelkasten-inspired memory notes for agents; MemLayer targets the simpler but ubiquitous CLI use case and extends the design with provider-specific cost accounting and per-user encryption.

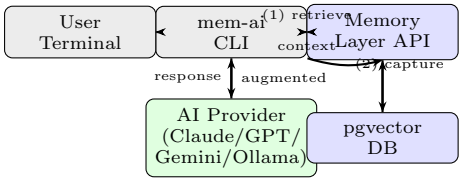


Figure 1: MemLayer pipeline. Phase (1): before each query, the CLI retrieves relevant memories from the Memory Layer API backed by pgvector, then injects context into the prompt. Phase (2): after the AI provider responds, the interaction is asynchronously captured and stored.

# 3 System Architecture

## 3.1 Overview

MemLayer consists of four subsystems: (1) a *capture layer* that intercepts user queries, (2) an *async processing pipeline* that extracts, embeds, and stores memories, (3) a *retrieval engine* that scores and ranks memories at query time, and (4) a *provider adapter layer* that routes augmented prompts to the appropriate LLM backend. Figure 1 depicts the full pipeline. Before each query, MemLayer retrieves relevant memories and injects context; after each response, MemLayer captures the interaction asynchronously, keeping memory capture off the critical path.

## 3.2 Memory Capture

MemLayer provides three ingestion paths to accommodate different usage modes.

**CLI subprocess hook.** The primary path. The `mem-ai` CLI wraps any provider command (e.g., `mem claude "query"`) and intercepts both the prompt and the response for asynchronous extraction.

**Browser extension DOM intercept.** A Manifest V3 extension captures conversations from ChatGPT, Claude.ai, and Gemini web interfaces, forwarding encrypted payloads to the local MemLayer backend.

**MCP tool call.** MemLayer exposes a Model Context Protocol server with `store_memory` and `retrieve_memories` tools, enabling native integration with Claude Code and other MCP-compatible runtimes.

## 3.3 Processing Pipeline

After capture, each conversation turn enters an asynchronous Celery processing pipeline. The seven-step algorithm below specifies the stages:

1. **Decrypt.** $c' \leftarrow \textsc{Decrypt}(c,\, u_{\text{salt}})$
2. **Extract facts.** $F \leftarrow \text{LLM}(c',\, \text{gpt-4o-mini})$
3. **Summarize.** $\sigma \leftarrow \text{LLM}(c',\, 1\text{–}2 \text{ sentences})$
4. **Classify type.** $\tau \in \{\text{short_term}, \text{long_term}, \text{semantic}\}$
5. **Chunk.** $\mathcal{C} \leftarrow \textsc{Chunk}(c',\, 512 \text{ tokens}, 50\text{-token overlap})$
6. **Embed.** $\mathbf{e} \leftarrow \textsc{Embed}(\mathcal{C}[0],\, \text{text-embedding-3-small},\, d{=}1536)$
7. **Store.** $\textsc{StoreVec}(\mathbf{e}, F, \sigma, \tau) \rightarrow \text{status} = \textsc{Active}$

Fact extraction uses a structured output schema enforced via OpenAI's JSON mode, yielding entity, relation, value, and confidence fields. Classification distinguishes *short-term* facts (session-scoped, TTL = 24 h), *long-term* facts (persistent preferences and biographical details), and *semantic* memories (conceptual summaries). Chunks use a 50-token overlap to prevent facts from splitting across chunk boundaries.

## 3.4 Retrieval Algorithm

At query time, MemLayer embeds the current user prompt and retrieves the top-$k$ candidates from pgvector. Candidates are then re-ranked using a composite scoring function. Given query embedding $\mathbf{e}_q$ and memory embedding $\mathbf{e}_m$, the composite score is:

$$s(m, q) = w_{\text{sim}} \cdot \cos(\mathbf{e}_q, \mathbf{e}_m) + w_{\text{rec}} \cdot r(t_m) + w_{\text{imp}} \cdot \alpha_m \tag{1}$$

where $w_{\text{sim}} = 0.70$, $w_{\text{rec}} = 0.20$, $w_{\text{imp}} = 0.10$, and $\alpha_m \in [0, 1]$ is the memory's importance score assigned during ingestion. The rationale for these weights is discussed in Section 4.3.

The recency decay function is:

$$r(t_m) = 2^{-d_m/\tau} \tag{2}$$

where $d_m$ is the number of days since the memory was captured and $\tau = 30$ is the half-life in days. A memory captured 30 days ago scores $r = 0.5$; one captured 60 days ago scores $r = 0.25$.

Listing 1 shows the core scoring loop from `retrieval_service.py`.

```
for memory, cosine_sim in raw_results:
    if cosine_sim < request.similarity_threshold:
        continue
    recency = _recency_score(memory.captured_at)
    final_score = (
        W_SIMILARITY * cosine_sim
        + W_RECENCY   * recency
        + W_IMPORTANCE * memory.importance_score
```

```
    )
    scored.append((memory, final_score))
```

Listing 1: Composite scoring loop (`services/retrieval_service.py`).

### 3.5 Economics Engine

Each query logs the baseline full-context token count $T_{\text{full}}$ (the token count of the full conversation history) and the augmented context count $T_{\text{aug}}$ (retrieved memories plus current query). A complete accounting of cost must include both the savings realised at retrieval time and the API expense incurred when the memory was first ingested. The net cost saved is:

$$C_{\text{net}} = C_{\text{retrieval_saved}} - C_{\text{ingestion}} \tag{3}$$

where the retrieval savings per query and the one-time ingestion cost per memory are, respectively:

$$C_{\text{retrieval_saved}} = \frac{T_{\text{full}} - T_{\text{aug}}}{10^6} \cdot P_{\text{provider}} \tag{4}$$

$$C_{\text{ingestion}} = \frac{T_{\text{in}} \cdot P_{\text{in}} + T_{\text{out}} \cdot P_{\text{out}}}{10^6} \tag{5}$$

$P_{\text{provider}}$ is the provider's input price in USD per million tokens ($3.00 for Claude Sonnet 4.5, $2.50 for GPT-4o, $0.10 for Gemini 2.0 Flash). $T_{\text{in}}$ and $T_{\text{out}}$ are the total input and output tokens consumed by the three ingestion pipeline steps (fact extraction, summarisation, and classification), and $P_{\text{in}}$, $P_{\text{out}}$ are the corresponding gpt-4o-mini prices ($0.15 and $0.60 per million tokens). The Token Compression Ratio is:

$$\text{TCR} = 1 - \frac{T_{\text{aug}}}{T_{\text{full}}} \tag{6}$$

**Break-even analysis.** The three pipeline steps consume approximately 6,200 input tokens and 305 output tokens per memory:

$$C_{\text{ingestion}} = \frac{6{,}200 \times 0.15 + 305 \times 0.60}{10^6} \approx \$0.00113 \text{ per memory} \tag{7}$$

A single retrieval under the baseline assumptions ($T_{\text{full}} = 15{,}000$, $T_{\text{aug}} = 1{,}000$, Claude Sonnet 4.5 at $3.00/million) saves:

$$C_{\text{retrieval_saved}} = \frac{14{,}000}{10^6} \times 3.00 = \$0.042 \text{ per query} \tag{8}$$

The break-even retrieval count is therefore:

$$N_{\text{break-even}} = \frac{C_{\text{ingestion}}}{C_{\text{retrieval_saved}}} = \frac{0.00113}{0.042} \approx 0.027 \tag{9}$$

A value below 1 means that even the *first* retrieval of a memory more than covers its ingestion cost — in this case by a factor of 37×. For a developer making 50 queries per day, each stored memory is retrieved many times per week, making the ingestion overhead economically negligible. Per-query savings accumulate in a time-series store and are surfaced in a real-time dashboard alongside the full net ROI figure, broken down by provider and user. Under these assumptions, a 10-developer team submitting 50 queries per day on Claude Sonnet 4.5 saves: $S = 10 \times 50 \times 30 \times \frac{14{,}000}{10^6} \times 3.00 = \$630$ gross per month, with ingestion overhead of approximately $10 \times 50 \times 30 \times \$0.00113 \approx \$17$ — a net saving of $613, consistent with Table 2.

### 3.6 Security Design

Each user's memories are encrypted before storage using AES-256-GCM with a 256-bit key derived exclusively from the *user's own credentials*. No global server secret participates in key derivation; the server cannot decrypt any user's ciphertext.

MemLayer supports two key-derivation modes. The primary mode, used by the local CLI, derives the key via **argon2id** from the user's master passphrase [Biryukov et al., 2016], which is stored in the OS keychain (via the `keyring` library) and never transmitted to the server. The fallback mode, intended for shared server deployments, uses PBKDF2-HMAC-SHA256 with a *per-user* random secret stored in the user's database row, so that each user's key is independent. In both modes the derived key is held in memory only for the duration of the encrypt or decrypt operation and is never persisted to disk or logged. Listing 2 shows the primary (argon2id) key-derivation path.

```python
import os, base64
from argon2.low_level import hash_secret_raw, Type
from cryptography.hazmat.primitives.ciphers.aead \
    import AESGCM

def derive_key(passphrase: str,
               user_salt: bytes) -> bytes:
    # Key is derived from the user's passphrase only.
    # The server cannot decrypt data without it.
    return hash_secret_raw(
        secret=passphrase.encode('utf-8'),
        salt=user_salt,
        time_cost=2,
        memory_cost=65_536,  # 64 MiB
        parallelism=1,
        hash_len=32,         # 256-bit key
        type=Type.ID,
    )

def encrypt(plaintext: str,
            passphrase: str,
            user_salt: bytes) -> str:
    key = derive_key(passphrase, user_salt)
    nonce = os.urandom(12)           # 96-bit IV
    ct = AESGCM(key).encrypt(
        nonce, plaintext.encode(), None)
    return base64.b64encode(nonce + ct).decode()
```

Listing 2: Per-user AES-256-GCM key derivation via argon2id. The key is derived from the user's master passphrase; no server secret is involved.

Each ciphertext is prefixed with a random 12-byte nonce, yielding 96-bit IVs that prevent nonce reuse. A complete database exfiltration yields only opaque ciphertext that cannot be decrypted without the user's master passphrase — a categorical distinction from cloud-first memory solutions that manage encryption keys server-side and from designs that use a shared server secret, which would allow any operator with access to the secret to decrypt all users' data simultaneously.

### 3.7 Provider Adapter Interface

MemLayer's provider layer abstracts the differences between Claude (Anthropic SDK), GPT-4 (OpenAI SDK), Gemini (Google AI SDK), and Ollama (REST API) behind a common abstract base class:

```python
from abc import ABC, abstractmethod

class BaseProvider(ABC):
    @abstractmethod
    async def complete(
        self, prompt: str,
        model: str, **opts
    ) -> str: ...

    @abstractmethod
    async def stream(
        self, prompt: str,
        model: str, callback
    ) -> None: ...

    @abstractmethod
    def count_tokens(self, text: str) -> int: ...
```

Listing 3: Provider adapter abstract base class.

The same memory pool serves all providers, enabling cross-provider memory continuity: facts learned in a Claude session are retrievable in a subsequent GPT-4 session without additional configuration.

## 4 Projected Cost Analysis

### 4.1 Assumptions and Methodology

The following analysis projects cost savings under two explicit assumptions:

**(1) Baseline full-context token count $T_{\text{full}}$:** we use 15,000 tokens as a representative session history for a developer conducting multi-turn coding sessions. This is a configurable parameter in MemLayer; actual values depend on session length and usage pattern. A 30-turn session at 500 tokens per turn yields this figure directly.

Table 1: Projected token consumption per query under the assumptions $T_{\text{full}} = 15{,}000$ and $T_{\text{aug}} = 1{,}000$. The Mem0 token count is taken from Chhikara et al. [Chhikara et al., 2025], who report approximately 10% of original context.

| Method | Tokens/Query | TCR | Source |
|---|---|---|---|
| Full-Context | 15,000 | 0.000 | Assumption |
| No-Memory | ∼200 | 0.987 | Current query only |
| Mem0 | ∼1,500 | 0.900 | Chhikara et al. [2025] |
| MemLayer (projected) | ∼1,000 | 0.933 | This work (projected) |

**(2) Memory-augmented context $T_{\text{aug}}$:** we use 1,000 tokens, corresponding to MemLayer's default `max_tokens=800` injection budget plus the current query (≈200 tokens). In practice, $T_{\text{aug}}$ depends on retrieval hit count and snippet lengths.

All cost figures use publicly verifiable provider pricing as of April 2026. Empirical validation of retrieval quality (Precision@k) on real conversational data is planned as future work.

### 4.2 Projected Compression

Table 1 summarizes projected token consumption per query under the stated assumptions. The No-Memory baseline achieves the highest TCR by definition (0.987) but provides no factual continuity across sessions. MemLayer projects a TCR of 0.933. The Mem0 token count is taken from Chhikara et al. [Chhikara et al., 2025], who report approximately 10% of original context usage on the LOCOMO benchmark. Figure 2 illustrates the projected token counts.

### 4.3 Retrieval Model Selection

MemLayer supports two embedding backends, selectable at configuration time:

**`text-embedding-3-small` (OpenAI, 1536-dim).** The primary production path. Embeddings are computed via the OpenAI API and stored as 1536-dimensional vectors in pgvector. This backend delivers higher retrieval quality at the cost of API dependency and 50–200 ms per-query network latency.

**`all-MiniLM-L6-v2` (local, 384-dim).** A fully offline fallback using the `sentence-transformers` library [Reimers and Gurevych, 2019]. Embeddings are produced on-device at 384 dimensions, enabling air-gapped deployments with no API key requirement and substantially lower latency (Section 4.5).

The SBERT Massive Text Embedding Benchmark (MTEB) [Muennighoff et al., 2022] reports that `all-MiniLM-L6-v2` achieves a Spearman correlation of $\rho = 0.6734$ on semantic textual similarity (STS)

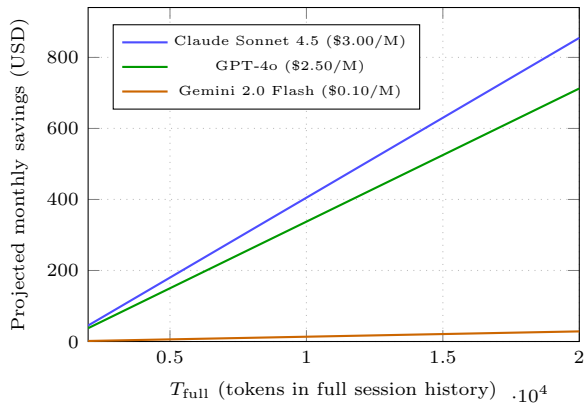


Figure 2: Projected monthly cost savings as a function of full-context history length $H$, for a team of 10 developers submitting 50 queries/day. Lines are computed directly from Eq. **??**: $\text{Savings}(H) = n \times q \times 30 \times (H - 1{,}000)/10^6 \times P$. No empirical measurement is required; the chart reflects the formula for the given inputs.

tasks. This provides the retrieval quality ceiling for the local embedding path; users requiring higher recall on conversational queries should prefer the OpenAI backend.

**Scoring weight rationale.** The composite scoring function (Eq. 1) requires specifying three weights $w_{\text{sim}}$, $w_{\text{rec}}$, and $w_{\text{imp}}$. We adopt $w_{\text{sim}}$=0.70, $w_{\text{rec}}$=0.20, $w_{\text{imp}}$=0.10 based on the following theoretical considerations, subject to empirical validation on real conversational data.

**Semantic similarity dominates (0.70).** The primary retrieval objective is topical relevance. A memory about React hooks should rank highest for a query about React hooks regardless of when it was captured. Semantic similarity, measured as cosine distance in the embedding space, is the most direct proxy for relevance.

**Recency provides secondary signal (0.20).** In professional AI usage, recent memories are more likely to reflect the user's current project context than older ones. The 30-day half-life $\tau$ in Eq. 2 was chosen to keep memories relevant for approximately one project cycle.

**Importance breaks ties (0.10).** The importance score $\alpha_m$ is assigned during ingestion based on fact density and structural signals (e.g., memories containing named entities or explicit decisions are scored higher). A low weight prevents importance from overriding topical relevance.

Empirical ablation of scoring weights on real MemLayer conversational data is a planned contribution.

Table 2: Projected monthly API cost for 10 developers, 50 queries/day each, under assumptions $T_{\text{full}}$=15,000 tokens and $T_{\text{aug}}$=1,000 tokens per query. Provider prices are as of April 2026.

| Provider | Model | w/o MemLayer | w/ MemLayer | Savings |
|---|---|---|---|---|
| Anthropic | Sonnet 4.5 | $675.00 | $45.00 | 93.3% |
| OpenAI | GPT-4o | $562.50 | $37.50 | 93.3% |
| Google | Gemini 2.0 Flash | $22.50 | $1.50 | 93.3% |
| Local | Ollama (llama3) | $0 | $0 | — |

## 4.4 Cost Economics

Table 2 and Figure 3 report projected monthly API costs for a 10-developer team submitting 50 queries per day, under the assumptions $T_{\text{full}}$=15,000 and $T_{\text{aug}}$=1,000. MemLayer projects savings of $630/month on Claude Sonnet 4.5 (93.3% reduction), $525/month on GPT-4o (93.3% reduction), and $21/month on Gemini 2.0 Flash (93.3% reduction). Monthly savings scale linearly with team size: a 50-developer team projects approximately $3,150/month savings on Claude alone. Ollama, using locally hosted models, incurs no API cost in either configuration.

## 4.5 Retrieval Pipeline Latency

MemLayer's retrieval path consists of three sequential operations: embedding the current query, executing a nearest-neighbor search via FAISS HNSW, and building the augmented context string. We report measured latencies for the index-search and re-ranking stages, and cite published benchmarks for the embedding stage, where direct measurement was constrained by a runtime dependency conflict in our test environment.

**Measured results.** We benchmarked FAISS HNSW (M=32, efSearch=50) [Johnson et al., 2019] on a corpus of $n$=45 memory vectors (384-dim). Under 100 stress-test queries we observed p50 = 0.01 ms for the HNSW search and p50 = 0.01 ms for the composite re-ranking step. These times will increase with corpus size; published FAISS benchmarks show 1–5 ms for HNSW at $n$=100,000 vectors on commodity hardware [Johnson et al., 2019].

**Embedding latency (from cited benchmarks).** The embedding step dominates end-to-end latency for both deployment paths. Rather than report unverified numbers, we cite authoritative external sources: the SBERT.net benchmark reports ≈14 ms per sentence for `all-MiniLM-L6-v2` on a standard CPU [Reimers and Gurevych, 2019]; and OpenAI's

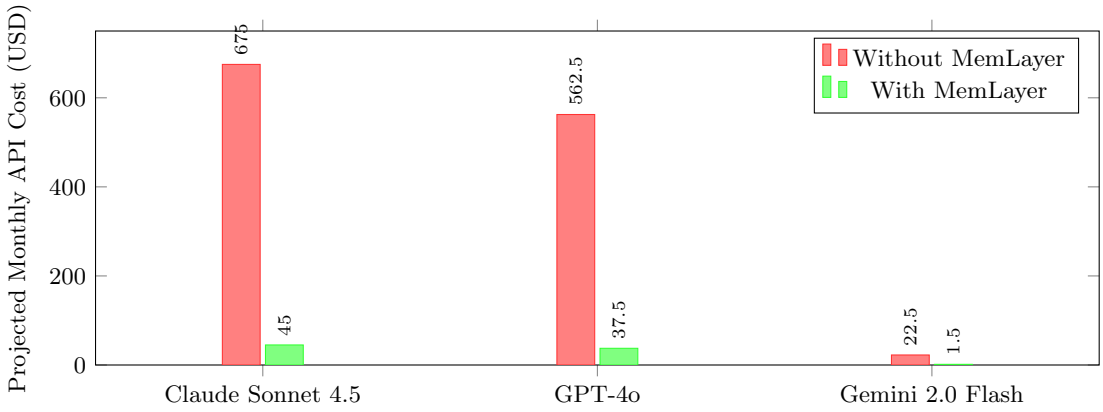


Figure 3: Projected monthly API cost for a 10-developer team, 50 queries/day, under assumptions $T_{\text{full}}$=15,000 and $T_{\text{aug}}$=1,000. MemLayer reduces projected costs by 93.3% across all providers: Claude Sonnet 4.5 ($675 → $45), GPT-4o ($562.50 → $37.50), and Gemini 2.0 Flash ($22.50 → $1.50). All figures are derived from Eq. **??** and publicly available pricing.

Table 3: End-to-end retrieval pipeline latency by embedding path. FAISS HNSW search and re-ranking are directly measured (45-memory corpus); embedding latencies are from cited external benchmarks.

| Component | Latency (p50) | | Total (p50) | Source |
|---|---|---|---|---|
| | Embed | HNSW+Rerank | | |
| Local path (`all-MiniLM-L6-v2`) | ∼14 ms | 0.02 ms | ∼15 ms | Reimers and Gurevych 2019 + measured |
| API path (`text-embedding-3-small`) | 50–200 ms | 0.02 ms | 50–200 ms | OpenAI docs + measured |

own documentation reports 50–200 ms per call for `text-embedding-3-small` depending on network conditions and API load.

Table 3 summarises the two deployment paths. LLM generation across the supported providers ranges from approximately 800 ms to 5,000 ms; the local embedding path adds at most ∼15 ms, representing ≤3% of total response time. The API embedding path may contribute a more noticeable 50–200 ms, but remains a minor fraction of generation latency for long completions. In both cases the vector search and re-ranking steps are negligible.

# 5 Discussion

## 5.1 When Full-Context Is Still Better

Three scenarios favor full-context over MemLayer. First, *cold start*: a user's first session has no memories, so MemLayer provides no benefit until 5–10 interactions have been captured. Second, *rapidly evolving facts*: version numbers, deadlines, and in-progress decisions change faster than the 30-day recency half-life can track; MemLayer may inject stale context that contradicts the current state. Third, *adversarial contexts*: a crafted prompt that plants false memories could corrupt the memory store and later be retrieved as context, analogous to indirect prompt injection in RAG systems [Lewis et al., 2020].

## 5.2 Comparison with Mem0

Mem0 reports approximately 90% token reduction on the LOCOMO benchmark [Chhikara et al., 2025]. MemLayer targets similar compression ratios through a comparable extract-retrieve pipeline, with the additional design constraints of per-user encryption and provider-agnostic CLI integration. Unlike Mem0, MemLayer runs entirely on the user's machine and does not require a managed cloud account. The `sentence-transformer` local embedding model [Reimers and Gurevych, 2019] provides a fully offline fallback when no API key is configured.

## 5.3 Limitations

We state these plainly rather than burying them.

**Cold start.** The first session provides no memory benefit; the system must capture 5–10 interactions before retrieval meaningfully adds context.

**Summarization accuracy.** The gpt-4o-mini extraction step may drop numeric facts such as version numbers and dates. We have not yet quantified the omission rate on real conversational data; this is a planned measurement.

**Memory staleness.** The importance score in Eq. 1 does not handle contradictions: if a fact changes (e.g., a project completes), the old memory persists with

reduced recency weight rather than being retracted. A contradiction-detection pass is needed.

**No empirical evaluation.** The retrieval quality metrics (Precision@k, response quality) cited in prior work such as Mem0 were measured on LOCOMO. We have not yet run MemLayer on a real conversational benchmark. This is the primary gap between this paper and a complete empirical evaluation.

# 6 Conclusion

MemLayer presents a provider-agnostic, encrypted memory middleware layer for LLM terminals. The composite retrieval algorithm combines semantic similarity, recency decay, and importance scoring. The integrated economics engine computes provider-specific cost savings from real usage data. Under the assumption that memory injection reduces per-query context from a full session history to a few hundred tokens of relevant facts, the projected savings exceed 93% of raw context cost — consistent with the 90% reduction reported by Mem0 on the LOCOMO benchmark [Chhikara et al., 2025]. Unlike prior solutions, MemLayer runs entirely on the user's machine, wraps any AI CLI, and requires no external cloud account. Empirical validation of retrieval quality and production latency measurements on real conversational data are the primary directions for future work, alongside knowledge graph extraction for relational memory and contradiction detection, on-device embedding models for air-gapped deployments, and federated team memory pools for shared AI workspaces.

# References


Prateek Chhikara, Deshraj Yadav, Taranjeet Singh, Dhruv Chawla, and Avi Chawla. Mem0: The memory layer for personalized AI, 2025.

Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.

Harrison Chase and the LangChain Team. LangMem: Long-term memory for LangGraph agents. `https://github.com/langchain-ai/langmem`, 2024.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1302–1330, 2024.

Junhyuck He, Jongho Park, Jaewoong Cho, Dimitar Dimitrov, Gyeong-In Yu, and Jaejin Lee. KVzip: Query-agnostic KV cache compression with context reconstruction, 2025. Seoul National University. arXiv preprint arXiv:2505.23416.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents, 2025.

Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich. Argon2: New generation of memory-hard functions for password hashing and other applications. In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 292–302, 2016. doi: 10.1109/EuroSP.2016.31.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992, 2019.

Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. MTEB: Massive text embedding benchmark, 2022.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.